

Enterprise AI Platform

Model Routing Strategy

The Elite 9-Layer Adaptive Routing Pipeline — complete technical walkthrough.

Scope. This document explains how the platform decides which LLM to call for each incoming request, how it validates the answer, and how it learns from every decision. It covers every layer of the pipeline in [src/layer0_model_infra/](#), from the trivial-query bypass to the asynchronous telemetry feedback loop.

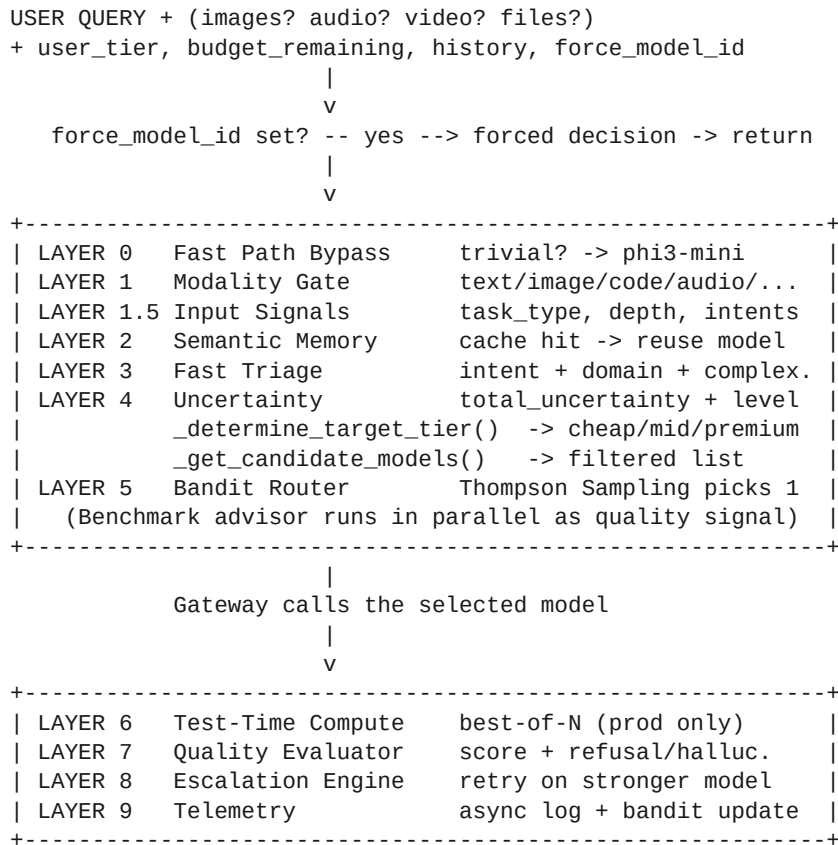
Audience. Engineers, reviewers and stakeholders who need to understand the routing decision chain in full — including the policy rules that override the learned policy.

Contents

#	Section	Layer
1	The big picture	—
2	Pipeline inputs	Entry
3	Fast Path Bypass	Layer 0
4	Modality Gate	Layer 1
5	Input Signal Extractor	Layer 1.5
6	Semantic Memory Cache	Layer 2
7	Fast Triage & Complexity Classifier	Layer 3
8	Uncertainty Estimator	Layer 4
9	Tier Selection & Candidate Filtering	Pre-bandit
10	Bandit Router (Thompson Sampling)	Layer 5
11	Benchmark Router (advisor)	Sidecar
12	Test-Time Compute	Layer 6
13	Quality Evaluator	Layer 7
14	Escalation Engine	Layer 8
15	Telemetry & Feedback Loop	Layer 9
16	Model Registry & Commercial Profiles	Sidecar
17	End-to-end worked example	—
18	Why this design	—

1. The big picture

The router orchestrator lives in `src/layer0_model_infra/router.py`. Entry point: `ModelRouter.route(query, ...)`. The pipeline runs every layer except when an upstream layer short-circuits the request — the Fast Path bypass and Semantic Memory cache are the two early exits.



The terms *layer* and *elite* here are not marketing — they are the formal names from `Synopsis_Layer0_Routing_System_V2`, which the implementation follows.

2. Pipeline inputs

`ModelRouter.route()` accepts the following parameters. Every layer below references these or values derived from them.

Parameter	Purpose
query	The user's text
has_images, has_audio, has_video, image_count, file_types, attachment_sizes_mb	Modality signals from the request
force_model_id	Manual model override. V07's manual-mode picker sets this; the pipeline still runs for logging but selection is skipped.
user_tier	free / standard / premium
budget_remaining	Fraction 0–1 of today's budget left
session_escalation_count	How many times this session has had to escalate
session_complexity_avg	Rolling avg query complexity for this session (0–1)
history	Prior conversation turns. Used to compute context-token density, which feeds the uncertainty estimator.
request_id	Telemetry correlation id (auto-generated if not provided)

Output. A [RoutingDecision](#) with the chosen model, ranked fallback list, full reasoning chain, estimated cost, confidence level and an escalation path.

3. Layer 0 — Fast Path Bypass

File: `src/layer0_model_infra/routing/fast_path.py`. A pure regex / keyword matcher that short-circuits the entire pipeline for trivially-handled queries so they don't pay the ~80–150 ms routing tax.

Trigger pattern	Recommended model	Confidence
Exact match in GREETINGS set (hi, hello, hey, ...)	ollama-phi3-mini	0.95
Exact match in ACKNOWLEDGMENTS (thanks, ok, bye, ...)	ollama-phi3-mini	0.95
Query < 15 chars with no '?' or '.'	ollama-phi3-mini	0.80
Regex match: <code>^what is \d+[\+\/]\d+, ^define \w+, etc.</code>	ollama-llama3.1-8b	0.85

On a hit, layers 1–9 are skipped. The decision is wrapped in `_create_fast_path_decision()`, which still calls the modality / triage / uncertainty components on the side — for telemetry only, not for selection.

4. Layer 1 — Modality Gate

File: `routing/modality_gate.py`. Determines *what kind of model* the query requires. Combines external attachment signals with content-based detection.

Output fields

```
class ModalityAnalysis:
    primary_modality: TEXT_ONLY | IMAGE | AUDIO | VIDEO | CODE | STRUCTURED
    requires_vision: bool          # has_images OR text mentions images/OCR/diagrams
    requires_audio: bool           # has_audio
    requires_code_model: bool      # code_density > 0.3 in the text
    token_count: int
    language: str                  # detected query language
```

Detection inputs

- External: **has_images**, **has_audio**, file MIME types, attachment sizes (MB).
- Text analysis: **_calculate_code_density()** counts backticks/fences/keywords.
- Structured-data scoring: **_detect_table_density()** flags pipe-tables, JSON, CSV.
- Language detection: statistical signals for Hindi, Spanish, etc.

This drives `_determine_model_type()` in the router: MULTIMODAL for vision, AUDIO for audio, otherwise TEXT. That's the first filter the registry will be queried with.

5. Layer 1.5 — Input Signal Extractor

File: [routing/input_signals.py](#). Pure heuristic feature extraction — no LLM calls. Produces ~20 structured fields read by the tier selection logic as **tie-breakers**.

Field	Type / range	What it captures
task_type	enum	QA GENERATION TRANSFORMATION ANALYSIS CONVERSATION CODING
requested_format	enum	Detected output shape (JSON / TABLE / CODE / FREE_TEXT)
requests_json / requests_table / requests_code / requests_detailed	bool	Explicit format asks from query keywords
instruction_count	int	Number of imperative verbs / numbered steps
has_multi_part	bool	Multiple distinct sub-questions
has_constraints	bool	Words like 'without X', 'exactly Y', 'at most N'
multi_intent_score	0–1	How many distinct intents coexist in one query
reasoning_depth	0–1	Density of 'why / because / derive / prove' tokens
numerical_reasoning_flag	bool	Math / statistics keywords present
code_generation_flag	bool	Coding-task keywords present
overall_difficulty	0–1	Aggregate heuristic score
These signals never override the LLM-classifier band on their own. They only tip the tier for <i>moderate-band</i> queries where the cheap-vs-mid decision is on a knife edge.		

6. Layer 2 — Semantic Memory Cache

File: [routing/semantic_memory.py](#). An **outcome-aware** cache keyed by semantic similarity. The bet: if 'What is *photosynthesis*?' got a good answer from a particular model 10 minutes ago, route a new 'Explain *photosynthesis*' to the same model without re-running the pipeline.

Lookup algorithm

- Normalize the query (lowercase, strip punctuation).
- Compute embedding similarity vs every cached entry.
- If **best_sim** $\geq$ **similarity_threshold** (0.85 dev / 0.90 prod), run validation guards.
- **Negation guard**: reject if one query has 'not' and the other doesn't.
- **Entity novelty**: reject if new named entities appear in the new query.
- **Outcome check**: only reuse entries whose previous result was successful.
- On pass: return **hit=True**, **matched_model_id**, **similarity**, **novelty=0**.
- On miss: still compute novelty = 1 – best_sim. Feeds the uncertainty estimator.

On a hit, the router calls [_create_cached_decision\(\)](#) and skips layers 3–5 (triage, uncertainty, bandit). Stale entries age out via [prune_stale_entries\(\)](#) at 30 days (dev) / 60 days (prod).

7. Layer 3 — Fast Triage & Complexity Classifier

Files: `routing/fast_triage.py` + `routing/complexity_classifier.py`. The brain of the pipeline.

Sub-stage A — Intent & Domain (heuristic)

Pure keyword-table lookups. Each function returns the chosen label plus a confidence score (0.68–0.98) based on how many keywords matched.

Output	Values
Intent	QA / CODING / PLANNING / CREATIVE / ANALYSIS / CASUAL / TECHNICAL / REASONING
Domain	TECH / BUSINESS / MEDICAL / LEGAL / SCIENCE / EDUCATION / CASUAL / GENERAL

MEDICAL and LEGAL are domain-sensitive — they immediately force the premium tier downstream as a hard safety rule, regardless of complexity or budget.

Sub-stage B — Complexity (LLM-based)

An actual LLM call via `litellm.completion` to a small, free model (`groq-llama-3.3-70b-free` in dev, `google/gemma-3-12b` in prod). The system prompt asks the model to score the query on a **5-dimensional rubric** mapped to Bloom's Taxonomy and Webb's Depth of Knowledge.

Dimension	Weight	What it measures
task_count	0.25	Atomic sub-tasks (single action → 10+)
domain_depth	0.20	Specialization (common → PhD)
reasoning_hops	0.25	Logical steps (lookup → multi-step proof)
output_structure	0.15	Output complexity (sentence → multi-component system)
knowledge_breadth	0.15	Topics to synthesize

From rubric to band

```
raw_score = sum(dim_value * weight)

raw_score < 0.12 -> trivial
    < 0.20 -> simple      (lowered from 0.30 after calibration)
    < 0.55 -> moderate
    < 0.85 -> complex     (raised from 0.80 to prevent expert over-routes)
    else -> expert
```

Confidence is derived, not self-reported

The system prompt explicitly forbids the model from reporting its own confidence. Two signals are combined post-hoc to produce a calibrated confidence:

- **Boundary distance** — queries far from band thresholds get high confidence; queries near a boundary get low confidence.

- **Rubric consistency** — if all 5 dimensions agree on a magnitude, confidence is high; wildly diverging dimensions mean low confidence.

If the LLM call fails or returns garbage JSON, the classifier falls back to a heuristic scorer with confidence capped at 0.50 — never higher than a band-boundary case.

8. Layer 4 — Uncertainty Estimator

File: [routing/uncertainty_estimator.py](#). Produces a **total_uncertainty** $\in [0, 1]$ from multiple sub-signals. This is *not* the same as classifier confidence: confidence asks "how sure are we about the band?"; uncertainty asks "how risky is misrouting this query?"

Sub-scores

Signal	What it detects
classification_entropy	Spread in intent / domain confidence
instruction_conflict	Incompatible commands (e.g. 'be brief' and 'very detailed')
context_dependency	Query needs prior context that is missing
cross_domain_score	Query spans 2+ domain vocabularies (legal + medical, tech + finance)
linguistic_uncertainty	Hedging language, vague phrasing
complexity_uncertainty	raw_score near a band boundary
domain_uncertainty	Domain confidence below threshold
context_uncertainty	Conversation density (ctx_token_density) high

The total is a weighted sum with **tier-dependent weights** — premium users get a different uncertainty profile than free users (they tolerate slower-but-safer routing). Output: [UncertaintyScore](#) with **total_uncertainty** and **confidence_level** $\in \{\text{HIGH, MEDIUM, LOW}\}$.

9. Tier Selection & Candidate Filtering

Before model selection, the router commits to a **tier**: cheap, mid or premium. This is the central cost/quality knob. Implemented in `_determine_target_tier()`.

Hard rules (always premium)

```
if modality.requires_vision:      return "premium" # safest multimodal tier
if domain in {medical, legal}:    return "premium" # high-risk domain
if cross_domain_score >= 0.35:    return "premium" # cross-domain query
if total_uncertainty >= 0.7:      return "premium" # high total uncertainty
```

Classifier-primary routing

```
if band == "expert":      return "premium"
if band == "complex":     return "premium"
if band == "trivial":     return "cheap"
if band == "simple":       return "cheap"
# moderate falls through to tie-breakers...
```

Moderate — conservative escalation + tie-breakers

- **complexity_confidence** < 0.55 → mid (don't trust a low-confidence moderate)
- **raw_score** within 0.05 of moderate-complex boundary → mid
- **numerical_reasoning** AND **domain** == science → mid
- **code_generation** AND **has_constraints** → mid
- **reasoning_depth** ≥ 0.55 → mid
- **domain** in {tech, business, education} → mid
- **user_tier** == 'premium' → mid (premium users get safer moderates)
- Otherwise → cheap (confident moderate)

Candidate filtering pipeline

Once the tier is fixed, `_get_candidate_models()` runs:

- **Registry lookup**: `list_models(model_type=..., only_active=True)`.
- **Capability filter**: keep CODING-capable models if code generation is needed; keep REASONING-capable if `reasoning_depth` ≥ 0.45.
- **Tier filter**: explicit `routing_tier` match; fallback to price-based heuristic (`output` ≥ \$0.015 = premium; `local` < 10B = cheap).
- **Free-API preference**: if `PREFER_FREE_API_PROVIDERS=true`, drop Ollama-only options when GROQ / GOOGLE / OPENROUTER / HUGGINGFACE / COHERE candidates exist.
- **Sort** by safety bias (REASONING-first for high-risk) and total cost.

If the filter chain produces an empty set, **ModelNotFoundError** is raised. This is the only fatal exit point in the routing pipeline.

10. Layer 5 — Bandit Router (Thompson Sampling)

File: `routing/bandit_router.py`. This is where the actual model is picked from the candidate list.

The model: contextual multi-armed bandit

Each (context, model_id) pair is an **arm**. Each arm has Beta-distribution parameters that get updated with every observed outcome.

```
alpha = 1.0    # successes + 1    (uniform prior Beta(1,1))
beta_ = 1.0    # failures  + 1

expected_reward = alpha / (alpha + beta_)
theta_sample    = random.betavariate(alpha, beta_)    # in (0, 1)
```

Context = bucketing key

`BanditContext.to_key()` hashes the following fields. Same query type / user profile maps to the same arm history; very different requests have independent arms.

- **intent, domain, complexity_band**
- **uncertainty_score, has_vision, has_code**
- **input_difficulty, has_multi_part, has_constraints**
- **user_tier, budget_remaining**
- **session_escalation_count, session_complexity_avg**

Selection algorithm

```
for each candidate model:
    arm = arms.get((ctx_key, model_id)) or BanditArm(alpha=1, beta_=1)
    theta = arm.sample_thompson()
    selected = argmax(theta)
```

Thompson Sampling naturally balances exploration and exploitation. High-confidence arms have spiked Beta distributions (consistent picks); low-data arms have flat distributions (near-random picks). New models get tried often; proven winners get picked often.

Reward shaping (applied later, by telemetry)

```
reward = quality_score - cost_penalty - latency_penalty
success = (reward > threshold) and (not escalated)

alpha += int(success)
beta_ += int(not success)
total_reward += reward
```

The bandit therefore learns *jointly* across quality, cost and latency. A fast cheap model with acceptable quality eventually beats a slow expensive model with only marginally better quality.

Special operating modes

- **Full exploitation:** when `exploration_rate` is 0 (or after warmup is complete), pick `argmax(expected_reward)` instead of sampling.
- **Domain safety lock:** in medical / legal contexts, sample only among REASONING-capable models.

- **Budget lock:** if budget_remaining is below a threshold, premium arms are excluded from the sample.

11. Benchmark Router (advisor, runs in parallel)

File: [routing/benchmark_router.py](#). **Advisory only**. Runs alongside the bandit but does not change the bandit's decision — its purpose is to provide a second opinion for telemetry-driven bandit tuning.

Two methods, in priority order

- **trained_gbc** — a Gradient Boosting Classifier trained on benchmark data plus sentence-transformer query embeddings. Loaded from [benchmark_router_model.joblib](#) if it exists.
- **benchmark_lookup** — rubric-weighted scoring fallback when no trained model is on disk.

How the lookup works

- Load [model_benchmarks.json](#) — quality scores for ~22 industry LLMs across MMLU, HumanEval, GSM8K, MATH, BBH, GPQA, etc.
- The 5-dimensional rubric from the complexity classifier maps to benchmark relevance weights (reasoning_hops → GSM8K + BBH; domain_depth → MMLU + GPQA; output_structure → HumanEval; ...).
- For each model: $\text{value_score} = \sum(\text{benchmark_score} \times \text{relevance_weight}) - \text{cost_penalty}$.
- Return the top scorer as [BenchmarkRouterResult](#) on [RoutingDecision.benchmark_recommendation](#).

The benchmark recommendation is logged alongside the bandit's pick. Telemetry comparing the two over time is the signal used to decide whether the bandit's learned policy needs re-warming or whether benchmark weights need re-tuning.

12. Layer 6 — Test-Time Compute

File: `routing/test_time_compute.py`. Enabled in production, disabled in dev via `enable_test_time_compute=False`.

For moderate-uncertainty queries — not high enough to trigger premium escalation, but not low enough for confident routing — the engine generates **N samples** from the selected model and picks the best one.

Strategy	How it picks the winner
BEST_OF_N	Score each sample with <code>QualityEvaluator</code> , return the highest scorer.
SELF_CONSISTENCY	Generate N at fixed temperature, return the majority-consensus answer.
VERIFICATION	Generate one, generate a verifier-style critique, regenerate with the critique in context.

Premium users get N = 3; standard users N = 2. Only fires when `should_use_ttc()` returns true — low confidence AND budget OK AND not in a refusal loop.

13. Layer 7 — Quality Evaluator

File: `routing/quality_evaluator.py`. Validates the response without necessarily calling another LLM — most of the signals are deterministic.

Multi-signal scoring

Signal	What it checks
JSON validity	If the response should be JSON, does it parse?
Code syntax	If Python, does <code>ast.parse()</code> succeed?
Format compliance	Did the model produce a table when one was asked for?
Schema validation	If a schema was provided, does the JSON match?
Completeness	Does the response address the query's instructions?
Coherence	Repetition / contradiction / fluency heuristics
Refusal detection	'I can't help with that' / 'I'm just an AI' / safety boilerplate
Hallucination risk	Confident factual claims with no supporting evidence pattern
Truncation	Cut off mid-sentence; incomplete code blocks

Returns `QualityScore` with **overall_quality** and a typed reason. An optional LLM-as-judge path is available — a Phi-4-mini-reasoning model rates accuracy / completeness / coherence / safety on 1–5 scales — but it's gated on cost.

14. Layer 8 — Escalation Engine

File: `routing/escalation_engine.py`. Climbs the **escalation path** (the candidate list sorted ascending by cost) when quality fails. Each retry runs on the next-most-expensive model.

Per-tier quality thresholds

```
ESCALATION_QUALITY_THRESHOLD = {  
    "free": 1.01,    # never escalate on quality alone (only refusals)  
    "standard": 0.65,  
    "premium": 0.75, # higher quality bar -> escalates sooner  
}
```

Refusal-detection always triggers escalation regardless of tier. Hard ceiling at `max_escalation_levels=3`. Each retry's cost and latency are rolled up into the cumulative bill that telemetry feeds back into the bandit reward.

15. Layer 9 — Telemetry & Feedback Loop

File: [routing/telemetry.py](#). The continuous-learning feedback path. Fire-and-forget — **never blocks the user request**.

RoutingTelemetry payload

A 25+ field dataclass: request_id, selected_model_id, final_model_id, intent, domain, complexity_band, quality_score, escalated, escalation_count, cost_usd, latency_ms, uncertainty_score, novelty_score, user_tier, budget_remaining, primary_modality, language, token_count, task_type, reasoning_depth, multi_intent_score, instruction_count, classification_entropy, cross_domain_score, ...

What the daemon thread does

- Persists the row to the **routing_telemetry** SQL table.
- Updates the matching bandit arm's α/β based on the observed reward.
- Calls [update_semantic_memory_entries\(\)](#) — records this query's outcome so the next similar query can short-circuit at Layer 2.
- Periodically calls [recalibrate_triage\(\)](#) to propose threshold updates when the complexity classifier drifts vs the gold set.
- [compute_evaluation_statistics\(\)](#) aggregates daily quality / cost / escalation rates for dashboards.

16. Sidecars — Model Registry & Commercial Profiles

Model Registry

File: `src/layer0_model_infra/registry.py` (~1050 lines). Source of truth for every model the platform can call.

Field	Purpose
<code>model_id</code>	Registry key, e.g. <code>groq-llama-3.3-70b-free</code>
<code>model_name</code>	String passed to LiteLLM, e.g. <code>groq/llama-3.3-70b-versatile</code>
<code>provider</code>	OPENAI / ANTHROPIC / GOOGLE / GROQ / OPENROUTER / HUGGINGFACE / COHERE / AZURE_OPENAI / LOCAL
<code>model_type</code>	TEXT / MULTIMODAL / AUDIO / EMBEDDING
<code>routing_tier</code>	CHEAP / MID / PREMIUM (used by tier filter)
<code>capabilities</code>	CODING / REASONING / FUNCTION_CALLING / VISION / ...
<code>pricing</code>	Input + output cost per 1K tokens (used everywhere for cost calc)
<code>is_active</code>	Soft toggle without deletion
<code>context_window,</code> <code>max_output_tokens</code>	Capacity limits

Commercial Profiles (V07 manual model picker)

When the V07 frontend's ModelSelector picks a manual model, it sends `simulation_profile_id` (for example `gpt-5.4-flagship`) to the backend. The profile is a **commercial-facing brand name** mapped to a *pool* of real backing models:

```
gpt-5.4-flagship -> premium pool:
    [groq-llama-3.3-70b-free, openrouter-gpt-4o, anthropic-claude-3-5-sonnet, ...]
```

The router sees `force_model_id` set (via the pool's first entry) and skips selection. Every other layer still runs for telemetry.

This is why the V07 chat header shows 'gpt-5.4-flagship' even though the actual API call is to Groq. The commercial profile is a presentation-layer abstraction over the registry.

17. End-to-end worked example

User submits: "Write a Python function to check if a string is a palindrome" — no attachments, standard tier, full budget.

```
Layer 0 (Fast Path):    no bypass (60+ chars, no greeting/ack match)
Layer 1 (Modality):     primary_modality=CODE, requires_code_model=True,
                        requires_vision=False, language='en'
Layer 1.5 (Signals):    task_type=CODING, code_generation_flag=True,
                        has_constraints=False, reasoning_depth=0.18,
                        instruction_count=1
Layer 2 (Memory):       miss, novelty_score=0.65
Layer 3 (Triage):
- intent=CODING (0.97), domain=TECH (0.9)
- rubric: task_count=0.3, domain_depth=0.4, reasoning_hops=0.5,
          output_structure=0.3, knowledge_breadth=0.2
- raw_score = 0.3*0.25 + 0.4*0.2 + 0.5*0.25 + 0.3*0.15 + 0.2*0.15 = 0.35
- 0.20 < 0.35 < 0.55 => band=MODERATE, confidence=0.78
Layer 4 (Uncertainty):  total_uncertainty=0.32 (LOW), cross_domain=0.05

Tier decision:
- Not vision; not medical/legal; cross_domain < 0.35; uncertainty < 0.7
- band == moderate, confidence (0.78) > 0.55, raw_score not near boundary
- code_generation_flag=True, but has_constraints=False -> tie-break skipped
- reasoning_depth (0.18) < 0.55
- domain=TECH -> "domain-sensitive moderate"
==> target_tier = MID

Candidates after filtering (TEXT + CODING + MID + free-API preference):
- groq-llama-3.3-70b-free
- openrouter-gemini-flash
- ollama-qwen3-8b   (kept only for gateway-level fallback)

Layer 5 (Bandit):
- context key = "intent=coding,domain=tech,complexity=moderate,..."
- theta samples: groq=0.74, gemini=0.68, qwen=0.62
- selected = groq-llama-3.3-70b-free

Benchmark advisor: groq-llama-3.3-70b-free with quality_score=0.82, cost=$0

Gateway call: litellm.acompletion(...) -> response in 380 ms

Layer 7 (Quality): code parses, query addressed -> overall_quality=0.86 PASS
Layer 8 (Escalation): not triggered
Layer 9 (Telemetry):
- log row: selected=groq, quality=0.86, latency=380, cost=$0.0001
- bandit update: (context, groq).alpha += 1
- semantic memory: record outcome for future hits
```

Total pipeline overhead: ~80–150 ms (dominated by the complexity-classifier LLM call). **Total cost:** ~\$0.0001. The expensive part is the model call itself, and the routing system chose the cheapest acceptable model from a 22-strong fleet.

18. Why this design

Property	How it is achieved
Cheap routing by default	Fast Path + free-API preference + cheap-tier selection for trivial / simple / confident-moderate
Safe routing for risky queries	Hard-rule premium for medical / legal / cross-domain / high-uncertainty / vision
Self-correcting	Quality evaluator + escalation engine retries on bad outputs (refusal, low score)
Self-learning	Thompson Sampling bandit + telemetry feedback loop continuously updates arm rewards
Calibrated complexity	LLM judge with 5-dimensional rubric, derived (not self-reported) confidence, boundary-margin escalation
Tiered escalation cost	Free tier never escalates on quality alone; premium escalates sooner
Auditability	Every decision carries the full reasoning chain, candidate list, sub-scores, and policy reason
Cost ceiling	Budget-aware tier downgrades, per-tier escalation limits, estimated cost on every decision

Single sentence summary. A hybrid policy: **hard safety rules** (vision, medical/legal, cross-domain, high uncertainty) override the learned policy. Within the safe zone, an **LLM-judged 5-dimensional rubric** sets the cost tier, a **Thompson Sampling bandit** picks the actual model, and **quality evaluation + escalation** closes the loop with telemetry-driven re-learning.